Inferring Aggregation Algebra and Minimal Mode Summaries

Mettapedia Working Notes

31 October 2026

Abstract

Five Lean modules connect the sorted run-count fold used in label propagation to histogram aggregation, implement a proof-producing classifier for additive counter updates, and characterize the smallest summary for two-label mode under future additions. Within a declared affine update language, admission is both sound and complete for translation updates and invariant under semantically equivalent spellings. The optimized runner preserves ordinary execution even on refusal. A separate contextual theorem shows that the two-label optional mode needs exactly a vote margin and an occupancy flag; fixing the empty default to the smaller label removes even the flag. These are source-model and information laws, not an arbitrary MeTTa algebra-discovery frontend or a native performance claim.

1 A bag function is not necessarily a homomorphism

A pure fold after sorting natural labels is insensitive to their original permutation. That makes its result a function of a bag. It does not make that result mergeable. The mode counterexample is small:

$$\mathrm{mode}[1,1,2] = \mathrm{mode}[1,1,1] = 1,$$

but appending $[2,2]$ makes the first winner 2 and leaves the second winner 1. Thus no binary operation on just the two partial winners computes every combined winner. Lean proves this impossibility universally over the proposed merge operation.

The correct decomposition is

$$\text{input collection} \xrightarrow{h} \text{mergeable summary} \xrightarrow{r} \text{answer}, \qquad h(x +\!\!+ y) = h(x) \oplus h(y).$$

For mode, h can be a histogram and r smallest-label argmax. The readout need not itself preserve $\oplus$. A useful region certificate therefore identifies the summary algebra and readout, as well as the allowed observations. The free commutative monoid property determines a *homomorphism* from its singleton action; it does not synthesize a suitable lifting for every bag function by itself.

This sits in the classical constructive-algorithmics tradition of list homomorphisms. Gibbons' third homomorphism theorem derives a homomorphism when the same list function has both leftwards and rightwards formulations [1]. The classifier below instead uses an independently proved matrix criterion for a specific update language. It is not a formalization of that general theorem.

2 Deriving the actual sorted run fold

The inspected CDLP source sorts the collected labels, then folds a tuple containing current label/count and best completed label/count. Equal labels increase the current count. A new

label finalizes the previous run, replacing the best only when its count is *strictly* larger. A final comparison closes the last run. Empty vote lists are handled by the enclosing source, which retains the vertex's old label.

`SortedRunMode` models those updates directly. Its invariant decomposes the processed list into

$$\text{completed runs} +\!\!+ [c, c, \ldots, c],$$

where the suffix length is the current count, every completed label is smaller than c, and the saved best is already the smallest modal label of the completed part. The proof of `append_run_best` establishes why a strict count increase is the right update: a tie must keep the earlier, smaller label. Induction through the fold proves the source result equals the independent histogram mode specification on every sorted input.

The mathematical optional label encodes a sentinel versus a natural label; its `none` does not mean an evaluator empty-answer event. The source's empty-input bypass and old-label fallback are proved separately.

`ModeAggregation` independently implements a counter-update loop and proves it computes list multiplicities. Histograms add pointwise across concatenation. A lexicographic score $(\text{count}, \text{reverse label})$ gives the greatest count and smallest label on ties, regardless of candidate enumeration order. The sparse finite-support interpretation has the same readout. Consequently,

$$\text{sourceMode}(\text{sort}(x)) = \text{readout}(\text{histogram}(x)).$$

Controls show that skipping sorting breaks the source run counter, changing strict comparison to non-strict comparison breaks ties, and forgetting multiplicity changes the winner.

3 A classifier that constructs its proof

`AdditiveFoldInference` provides a finite, pure update language for n natural-valued registers. Expressions contain register references, input-only weights, addition and multiplication by natural constants. Weights contain constants, the input value, equality indicators, addition and constant scaling. All expressions are total; effectful calls are not hidden inside the weight syntax.

Normalization constructs a coefficient vector and a residual input expression:

$$\text{eval}(e_i; s, x) = \sum_j A_{ij} s_j + w_i(x).$$

The algorithm checks whether $A = I$. On acceptance it returns the extracted weights and constructs a Lean certificate of

$$\text{step}(s, x) = s + w(x).$$

The theorem is not an input supplied by the caller. It follows from recursive normalization and the computed finite matrix check.

The criterion is exact within this regime:

$$A = I \quad \Longleftrightarrow \quad \forall s, x : \ \text{step}(s, x) = s + \text{step}(0, x).$$

Necessity follows by probing zero and each basis state. Therefore programs with identical transition semantics inside this IR have identical admission status, even when reassociation, zero-scaled registers or harmless extra terms disguise their source form. A control with zero-scaled state and reordered indicator terms is accepted. A decimal accumulator $s \leftarrow 10s + x$ is refused and

demonstrably order-sensitive. Refusing a program proves only that it is outside this translation regime, not that no other algebra can optimize it.

For accepted programs, the derived summary is

$$H(x_1 \cdots x_m) = \sum_{k=1}^{m} w(x_k), \qquad \mathrm{run}(s_0, x) = s_0 + H(x).$$

It is a bag homomorphism; any pure readout inherits permutation invariance. The executable inferred runner chooses this summary when admitted and the ordinary fold otherwise. `runInferred_exact` proves unconditional equality with ordinary execution for every program of the IR, and the runner composes over sequential input segments. Refusal is not an error or an empty result.

This is proof-producing inference for a declared family, not automatic extraction from arbitrary MeTTa functions. The CDLP sorted-fold-to-histogram translation is a separate proved transformation. Its discovery has not been delegated to an unverified finite testing oracle.

4 Grouped execution and label-domain admission

The histogram update $s_i \leftarrow s_i + [x = i]$ is accepted by the classifier for every finite register domain. `GroupedModeRegion` extends the resulting counter vector by zero outside that domain and proves exact readout when every actual label lies inside it. An explicit negative control shows that ignoring an out-of-domain label loses the answer.

The existing `JoinAggregation.groupFold` supplies one product-monoid fold over group keys. Projection at each key equals folding its filtered vote list, and partial group summaries merge pointwise. The joined theorem connects this carrier to the sorted source mode, including ties and duplicate solutions. It preserves the aggregate of the actual collected occurrences, without assuming graph-incidence multiplicities from a source comment.

This licenses a mathematical count-plus-argmax region. Native tuple layout, counter overflow, state revision, publication of the next label generation, source return counts and concurrent readers are further refinement obligations. The proof does not replace CDLP's outer iteration schedule.

5 A smaller summary derived from future observations

For two labels $\mathsf{false} < \mathsf{true}$, let

$$D(x) = \#\mathsf{true}(x) - \#\mathsf{false}(x), \qquad E(x) = [x \neq []].$$

The summary (D, E) combines as

$$(d, e) \oplus (d', e') = (d + d',\ e \vee e').$$

An occupied summary returns true exactly when $d > 0$; an unoccupied summary returns no modal label. The readout is proved equal to independent counting and then connected to the original sorted mode via labels 0 and 1. The reachable summaries are exactly those where unoccupied implies zero margin; combining reachable summaries preserves this invariant.

The stronger result is continuation minimality. Define two inputs to be equivalent when every possible future added suffix produces the same optional modal answer. Then Lean proves

$$\big(\forall z,\ \mathrm{mode}(xz) = \mathrm{mode}(yz)\big) \quad \Longleftrightarrow \quad (D(x), E(x)) = (D(y), E(y)).$$

To distinguish unequal margins, add votes that bring one margin exactly to zero and the other across the tie boundary. Every integer difference has an explicit nonempty vote-list realization. Empty and balanced nonempty inputs are separated by the empty future suffix. Every summary able to answer all future additions must distinguish these classes.

Thus a list with one vote of each kind and a list with two votes of each kind have the same future-addition behavior. This is not permission to forget counts universally. Deleting one vote of each kind empties the first but leaves the second nonempty. Full histograms have a separately proved one-occurrence erase law; the additive merge theorem alone does not establish deletion maintenance. The same summary cannot recover total vote count: the two example lists have lengths two and four. A source count, evidence multiplicity or work-accounting observer needs additional retained information.

Even occupancy is observer-relative. If the caller fixes the empty default to false, the least label, both an empty and a balanced nonempty input return false. In that profile,

$$(\forall z,\ \mathrm{defaultFalse}(xz) = \mathrm{defaultFalse}(yz)) \quad \Longleftrightarrow \quad D(x) = D(y).$$

A true empty default separates the previously merged empty/balanced cases. The known caller context therefore changes which information can be erased.

Finally, no finite-state summary can answer all future additions on unbounded inputs: arbitrarily many integer margins have distinguishing contexts. A fixed number of integer fields is not a fixed number of bits. The result is an exact semantic information boundary, not a universal constant-space implementation.

6 Scope and next refinements

The implemented classification is complete for additive translation semantics inside the affine IR and incomplete as a classifier of arbitrary programs. The minimality theorem concerns two-label mode, not arbitrary many-label modes. No theorem states that the fastest implementation follows automatically from a semantic description. Admission supplies permission; physical cost models and measurements must choose an implementation.

The next refinements are a source/profile extractor with revision and effect certificates; broader summary inference for run folds and other algebraic families; larger-domain continuation quotients; and native grouped execution with accounting, lifetime and cost correspondence. Metamorphic tests should vary equivalent source spellings and deliberately change effects, multiplicity, tie policy, label domains and observable boundaries.

The modules under `Mettapedia/Machines/` are `ModeAggregation`, `SortedRunMode`, `AdditiveFoldInference`, `GroupedModeRegion`, and `BinaryModeInformation`. `AggregationInferenceAudit` inventories their checked declaration bodies and transitive primitive axioms.

References

[1] Jeremy Gibbons, "The Third Homomorphism Theorem," *Journal of Functional Programming* 6(4), 657–665, 1996. doi:10.1017/S0956796800001908.